

Reporte de Arquitectura y Viabilidad: Migración a Streamlit

Este documento detalla el análisis de viabilidad, la propuesta de arquitectura y el diseño técnico para reemplazar la interfaz de consola actual de [src/app.py](file:///home/rdev/Documentos/3_proyecto/2_AgentePOO/src/app.py) por una aplicación web interactiva utilizando la biblioteca **Streamlit**.

1. Arquitectura de la Solución

El principal reto de integrar un framework reactivo como Streamlit con un agente RAG es adaptar el flujo de ejecución. Streamlit ejecuta todo el script de arriba a abajo en cada interacción (rerun). A continuación, se detalla cómo estructurar la interacción con [src/agent.py](file:///home/rdev/Documentos/3_proyecto/2_AgentePOO/src/agent.py) sin perder la lógica stateless y sin inflar el consumo de tokens:

A. Persistencia del Historial de Chat (`st.session\_state`)

Para mantener el historial de chat entre ejecuciones consecutivas sin requerir bases de datos externas, utilizaremos el diccionario de estado nativo de Streamlit:

```
`st.session_state`.
```

* Guardaremos la conversación en una lista limpia bajo `st.session\_state.messages`.

* Cada elemento de esta lista constará de la estructura básica: `{"role": "user" | "assistant", "text": str}`.

B. Prevención de la Inflación de Tokens (Historial Limpio RAG)

Para no sobrecargar el modelo de lenguaje de Google Gemini en cada turno:

1. **Historial Limpio:** En `st.session\_state.messages` se almacena exclusivamente el diálogo de entrada del usuario y la respuesta de texto del asistente. **No** se almacenan los fragmentos de contexto recuperados (chunks).
2. **Contexto Efímero:** Al igual que en la lógica actual de [src/agent.py](file:///home/rdev/Documentos/3_proyecto/2_AgentePOO/src/agent.py), el fragmento recuperado mediante ChromaDB (`contexto\_formateado`) se formatea en un prompt final que se inyecta **únicamente** en el turno actual.
3. **Llamadas Stateless:** La lista final enviada al SDK de Gemini (`client.models.generate\_content`) se reconstruye en tiempo de ejecución mapeando los mensajes limpios del historial más el turno actual extendido con su respectivo contexto RAG. De este modo, los contextos de búsquedas pasadas se descartan del historial conversacional enviado al modelo, garantizando que el consumo de tokens no se infle exponencialmente.

```
```mermaid
```

```
graph TD
```

```
 User([Usuario ingresa pregunta]) --> Input[st.chat_input]
 Input --> RenderUser[Mostrar pregunta en pantalla]
 Input --> HyDE[Optimizar con HyDE]
 HyDE --> Chroma[(ChromaDB)]
 Chroma --> Retrieve[Recuperar Contexto Relevante]
 Retrieve --> BuildPayload[Construir Historial Limpio + Contexto actual]
 BuildPayload --> GeminiAPI[Llamada Stateless a Gemini]
 GeminiAPI --> RenderBot[Mostrar respuesta del Asistente]
 RenderBot --> SaveState[Guardar en st.session_state sin Contexto]
```

```
```
```

C. Reutilización del Índice Semántico

La función

[crear_retriever_desde_pdf](file:///home/rdev/Documentos/3_proyecto/2_AgentePOO/src/vectorstore.py#L83) lee el archivo PDF de la carpeta `data/` y carga o genera el índice en `chroma\_db/`. Cargar esta base de datos en cada rerun ralentizaría drásticamente la interfaz.

Para resolver esto, usaremos el decorador `@st.cache\_resource` de Streamlit sobre la función de inicialización:

```
```python
@st.cache_resource
def cargar_retriever():
 retriever, _ = crear_retriever_desde_pdf()
 return retriever
```
```

De esta manera, ChromaDB se inicializa y se lee de disco una única vez al levantar el servidor de Streamlit y se comparte en memoria para todas las interacciones subsiguientes.

2. Gestión de Estilos y Colores

Streamlit soporta tematización nativa de manera robusta a través de un archivo de configuración ubicado en `.streamlit/config.toml`. Esto evita la inyección de bloques CSS complejos o inseguros y garantiza que los componentes nativos (`st.chat\_message` y `st.chat\_input`) se adapten coherentemente a los modos claro y oscuro del sistema.

Configuración propuesta para `.streamlit/config.toml`

```
```toml
[theme]
Color de acento primario (utilizado para loaders, foco de inputs y botones)
primaryColor = "#007BFF"

Color del fondo de la aplicación (optimizado para modo oscuro por defecto)
backgroundColor = "#0F172A"

Fondo secundario (utilizado para las tarjetas de mensajes y barras laterales)
secondaryBackgroundColor = "#1E293B"

Color del texto principal para garantizar un contraste óptimo
textColor = "#F8FAFC"

Tipografía limpia y moderna
font = "sans serif"
```
```

Comportamiento Dinámico Claro / Oscuro:

* **Uso de Variables del Sistema:** Streamlit detecta la preferencia de tema del navegador/SO del usuario. Si el usuario cambia manualmente al modo claro en el menú de Streamlit, el framework recalcula y suaviza la paleta utilizando colores coherentes (por ejemplo, adaptando las burbujas secundarias a grises claros) para mantener la legibilidad y el contraste de accesibilidad (WCAG AA).

* **Sin CSS hacks:** Evitar la inyección de CSS mediante `st.markdown("<style>...", unsafe\_allow\_html=True)` asegura la compatibilidad futura de la aplicación, pues las clases internas de Streamlit cambian con frecuencia entre versiones.

3. Nivel de Dificultad

Calificación: 15% (Complejidad Muy Baja - Baja)

Justificación:

1. **Lógica de negocio intacta:** Los archivos existentes de carga ([src/loader.py](file:///home/rdev/Documentos/3_proyecto/2_AgentePOO/src/loader.py)), segmentación

([src/splitter.py](file:///home/rdev/Documentos/3_proyecto/2_AgentePOO/src/splitter.py)), indexación ([src/vectorstore.py](file:///home/rdev/Documentos/3_proyecto/2_AgentePOO/src/vectorstore.py)) y configuración ([src/config.py](file:///home/rdev/Documentos/3_proyecto/2_AgentePOO/src/config.py)) ya son completamente modulares. No requieren ninguna alteración para funcionar en el entorno web.

2. ****Refactorización mínima del Agente:**** El bucle de consola infinito (``while True``) presente en

[iniciar_agente_interactivo](file:///home/rdev/Documentos/3_proyecto/2_AgentePOO/src/agent.py#L43) es el único componente que no se puede importar directamente a Streamlit tal como está. Sin embargo, su lógica de negocio secuencial (HyDE -> Retriever -> Construcción de Payload -> Gemini) es fácilmente representable en un flujo de ejecución lineal adaptado a Streamlit.

3. ****API Nativa de Chat:**** Streamlit ofrece una experiencia visual de chat madura e interactiva lista para usar. Implementar esta lógica requiere menos de 70 líneas de código limpio en un nuevo archivo (por ejemplo, ``src/app_web.py``).

4. ****Despliegue e Integración en OCI:**** La lógica de lectura de variables de entorno y rutas en [src/config.py](file:///home/rdev/Documentos/3_proyecto/2_AgentePOO/src/config.py) (usando ``os.environ`` y ``Path(__file__)``) es compatible de forma nativa con despliegues web en contenedores o instancias virtuales en Oracle Cloud Infrastructure (OCI). Solo es necesario exponer el puerto web estándar de Streamlit (8501).

4. Boceto de Código de la Interfaz

A continuación se presenta la estructura de lo que sería el nuevo punto de entrada web (``src/app_web.py``). Este código no bloquea la ejecución, mantiene la coherencia semántica del RAG e implementa la optimización HyDE:

```
```python
import streamlit as st
from google import genai
from google.genai import types

Importaciones de la lógica de negocio actual
from config import MODEL_NAME, MAX_OUTPUT_TOKENS, TEMPERATURE, obtener_api_key
from vectorstore import crear_retriever_desde_pdf
from agent import generar_documento_hipotetico

1. Configuración básica de la aplicación
st.set_page_config(
 page_title="Asistente Académico POO",
 page_icon="",
 layout="centered"
)

st.title("Asistente de Programación Orientada a Objetos")
st.caption("Consulta el material académico del curso utilizando Inteligencia Artificial.")

2. Inicialización de recursos compartidos (Cache para evitar reruns lentos)
@st.cache_resource
def inicializar_recursos():
 """Carga el índice ChromaDB y valida la API Key de Gemini."""
 retriever, estado_db = crear_retriever_desde_pdf()
 api_key_gemini = obtener_api_key()

 if not api_key_gemini:
```

```

 st.error("✖Error de configuración: No se detectó GEMINI_API_KEY en el entorno.")
 st.stop()

 client = genai.Client(api_key=api_key_gemini)
 return retriever, client, estado_db

try:
 retriever, client, estado_carga = inicializar_recursos()
 # Mostramos el estado del índice de forma discreta en la barra lateral
 st.sidebar.success(f"Configuración: {estado_carga}")
except Exception as error:
 st.error(f"✖Error al iniciar el índice: {error}")
 st.stop()

3. Inicialización del historial en session_state
if "messages" not in st.session_state:
 st.session_state.messages = []

4. Renderizar el historial conversacional existente
for message in st.session_state.messages:
 with st.chat_message(message["role"]):
 st.markdown(message["text"])

5. Captura y procesamiento de nuevos mensajes
if pregunta_usuario := st.chat_input("Escribe tu pregunta sobre el documento..."):
 # Renderizar el mensaje de inmediato
 with st.chat_message("user"):
 st.markdown(pregunta_usuario)

 # Procesar la respuesta con el spinner de carga
 with st.chat_message("assistant"):
 with st.spinner("Pensando..."):
 # A. Optimización de consulta (HyDE)
 pregunta_mejorada = generar_documento_hipotetico(client, pregunta_usuario)

 # B. Búsqueda semántica en el PDF indexado
 documentos_relevantes = retriever.invoke(pregunta_mejorada)
 contexto_formateado = "\n---\n".join(
 doc.page_content for doc in documentos_relevantes
)

 if not contexto_formateado.strip():
 respuesta_texto = "No encuentro esa información en el documento."
 else:
 # C. Construir secuencia de turnos stateless para evitar inflación de tokens
 contents = []
 for msg in st.session_state.messages:
 contents.append(
 types.Content(
 role=msg["role"],
 parts=[types.Part.from_text(text=msg["text"])]
)
)

 # Inyectamos el RAG exclusivamente en el prompt final del turno activo
 prompt_final = (
 "Usa SOLO el contexto siguiente si realmente sirve para responder.\n\n"
 f"CONTEXTO DEL DOCUMENTO:\n{contexto_formateado}\n\n"
 f"PREGUNTA DEL USUARIO:\n{pregunta_usuario}\n\n"

```

```

 "Si la respuesta no está en el contexto, responde exactamente: "
 "No encuentro esa información en el documento."
)
 contents.append(
 types.Content(
 role="user",
 parts=[types.Part.from_text(text=prompt_final)]
)
)

D. Configuración y llamada de inferencia a Gemini
system_instruction = (
 "Eres un asistente académico experto en el documento proporcionado. "
 "Responde solo con información respaldada por el contexto. "
 "Si la respuesta no está en el texto, dilo claramente. "
 "Sé claro, breve y preciso."
)

config = types.GenerateContentConfig(
 system_instruction=system_instruction,
 max_output_tokens=MAX_OUTPUT_TOKENS,
 temperature=TEMPERATURE,
)

try:
 response = client.models.generate_content(
 model=MODEL_NAME,
 contents=contents,
 config=config
)
 respuesta_texto = response.text
except Exception as api_error:
 respuesta_texto = f"É Ocurrió un error con la API de Gemini: {api_error}"

Mostrar la respuesta final
st.markdown(respuesta_texto)

E. Guardar en el historial limpio local de la sesión (evitando re-escribir el RAG)
st.session_state.messages.append({"role": "user", "text": pregunta_usuario})
st.session_state.messages.append({"role": "assistant", "text": respuesta_texto})

```